

Bienvenido a NOVA_sense

¿Qué es NOVA_sense?

NOVA_sense es una pequeña placa electrónica que le da **sentidos** a tus proyectos. Así como tú usas tus ojos para ver y tu oído interno para mantener el equilibrio, NOVA_sense le permite a un robot o cualquier dispositivo electrónico **saber hacia dónde apunta, detectar movimiento y encontrar el Norte**, todo desde una sola placa del tamaño de una moneda.

Piensa en ella como el **“cerebro sensorial”** de tu proyecto: tú le das las instrucciones, y ella te devuelve información sobre lo que está pasando en el mundo real.

¿A qué se parece? — Comparaciones sencillas

Sensor en NOVA_sense	¿A qué se parece en la vida real?	¿Qué detecta?
Magnetómetro (LIS2MDL)	Una brújula digital como la de tu celular	Hacia dónde está el Norte magnético
Acelerómetro (dentro del LSM6DS3)	El sensor que gira la pantalla de tu celular cuando lo inclinas	Inclinación, vibración, caídas
Giroscopio (dentro del LSM6DS3)	El sensor que detecta giros cuando juegas un videojuego moviendo el celular	Velocidad y dirección de rotación

En resumen: tu celular usa sensores muy parecidos a los de NOVA_sense para rotar la pantalla, orientar mapas y detectar movimiento. NOVA_sense te da esos mismos poderes para tus propios inventos.

¿Qué puedes hacer con ella?

- **Construir un robot que sepa hacia dónde va** — como un auto a control remoto que siempre sabe dónde está el Norte.
- **Detectar movimiento y gestos** — inclinar, agitar, girar: ideal para controles de juegos o instrumentos interactivos.
- **Crear una brújula electrónica** — muestra en una pantalla hacia dónde apuntas, igual que la app de brújula de un teléfono.
- **Registrar datos de movimiento** — grabar cómo se mueve un objeto (por ejemplo, una bicicleta o un dron) y analizarlo después.
- **Detectar la presencia de imanes o metales cercanos** — gracias a la sensibilidad del magnetómetro.
- **Estabilizar mecanismos** — como un dron que se auto-nivela o una cámara que compensa los temblores de la mano.

Antes de empezar

Este manual está escrito en un lenguaje sencillo para que cualquier persona, sin importar su experiencia, pueda seguir los pasos y lograr resultados. Sigue los ejemplos en orden, prueba cada uno y, si algo no funciona, revisa las conexiones antes de continuar.

1 Conoce tu NOVA_sense

1.1 Descripción general

En esta página se muestran los componentes principales del módulo NOVA_sense con su nombre común y una breve descripción para facilitar su identificación.

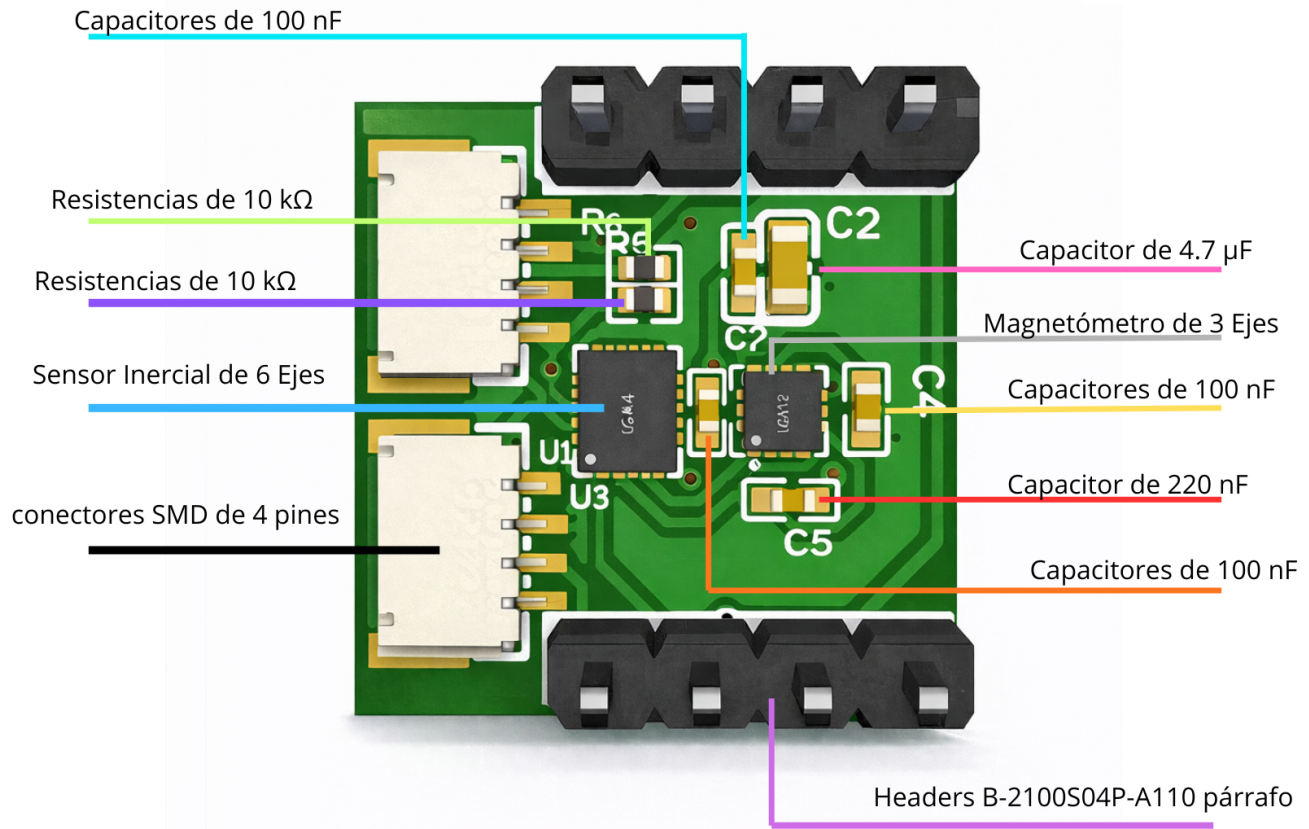


Figure 1: Componentes NOVA_sense

1.2 Componentes (nombre común — descripción)

- **IMU (LSM6DS3)** — Acelerómetro y giroscopio integrados (6 ejes). Mide aceleración lineal y velocidad angular para detección de movimiento y fusión inercial.
- **Magnetómetro (LIS2MDL)** — Sensor de campo magnético 3 ejes utilizado como brújula electrónica para orientación respecto al Norte magnético.
- **Conectores 4 pines (CN1, CN2)** — Conectores SMD para alimentación y señales (paso 1.0 mm) usados para integrar el módulo en placas madre o accesorios.
- **Condensadores 100 nF / 220 nF / 4.7 μ F** — Filtrado y estabilización de la alimentación.
- **Resistencias 10 k Ω** — Pull-ups/pull-downs para asegurar niveles lógicos estables en líneas digitales.

1.3 Resumen funcional

Todos estos componentes trabajan juntos para proporcionar el soporte necesario para integrarse fácilmente en proyectos de robótica, navegación y registradores de datos.

2 Hardware y Conexiones

NOVA_sense se comunica con cualquier microcontrolador (Arduino, Raspberry Pi Pico, ESP32, etc.) mediante **I2C**, un protocolo que solo necesita **4 cables**: alimentación (3.3 V), tierra (GND) y dos cables de datos (SDA y SCL). Si ya has conectado otro sensor I2C antes, NOVA_sense se conecta exactamente igual — pueden compartir los mismos cables.

2.1 Conectar la NOVA_sense a cualquier microcontrolador

La NOVA_sense se comunica por I2C y necesita alimentación de 3.3 V. Aquí explicamos en términos simples cómo conectarla usando el microcontrolador que tengas (Arduino, Raspberry Pi Pico/NOVA_pico, ESP32, etc.) y qué herramientas puedes usar para programar y leer datos.

Conexiones eléctricas:

- **Alimentación:** conecta 3V3 de tu microcontrolador al pin de 3.3V de la NOVA_sense y conecta GND <-> GND (masa). Nunca alimentes la placa con 5V en los pines de señal.
- **I2C (datos):** conecta SDA <-> SDA y SCL <-> SCL entre tu microcontrolador y la NOVA_sense. Si tu placa tiene varios pines I2C, usa los que indique su documentación.

Notas de cableado:

- Asegura GND común entre ambas placas.
- Añade una resistencia de pull-up si tu bus I2C no las incluye (muchas placas ya las llevan integradas).

2.2 Programas y entornos para escribir código

Puedes usar casi cualquier editor o IDE que soporten tu microcontrolador. Ejemplos comunes:

Con estos programas podrás escribir código que use el bus I2C para leer registros del sensor y así obtener orientación o aceleración.

2.3 Leer registros y memoria del sensor (explicación simple)

Lo habitual es leer registros del sensor por I2C: cada sensor tiene direcciones de registro (por ejemplo WHO_AM_I) que te permiten comprobar que el chip responde. Para obtener datos, lees los registros de salida y conviertes los bytes en valores utilizables (ver sección de programación).

Si por “leer la memoria del NOVA_sense” te refieres a acceder a archivos o programas guardados en una placa que controle la NOVA_sense (por ejemplo la NOVA_pico), eso se hace desde el entorno de programación del microcontrolador (Thonny, gestor de archivos del IDE, o mediante conexión USB/serial). No se modifica la memoria del sensor directamente; en su lugar se leen registros vía I2C.

2.4 La NOVA_pico como controlador

La NOVA_pico es una placa compatible con RP2 que puede ejecutar MicroPython.

Cómo usar ambas juntas (resumen):

- Conecta 3V3 <-> 3V3 y GND <-> GND entre NOVA_pico y NOVA_sense.
- Conecta SDA y SCL de la NOVA_pico a SDA y SCL de la NOVA_sense.
- Instala MicroPython en la NOVA_pico y abre Thonny.

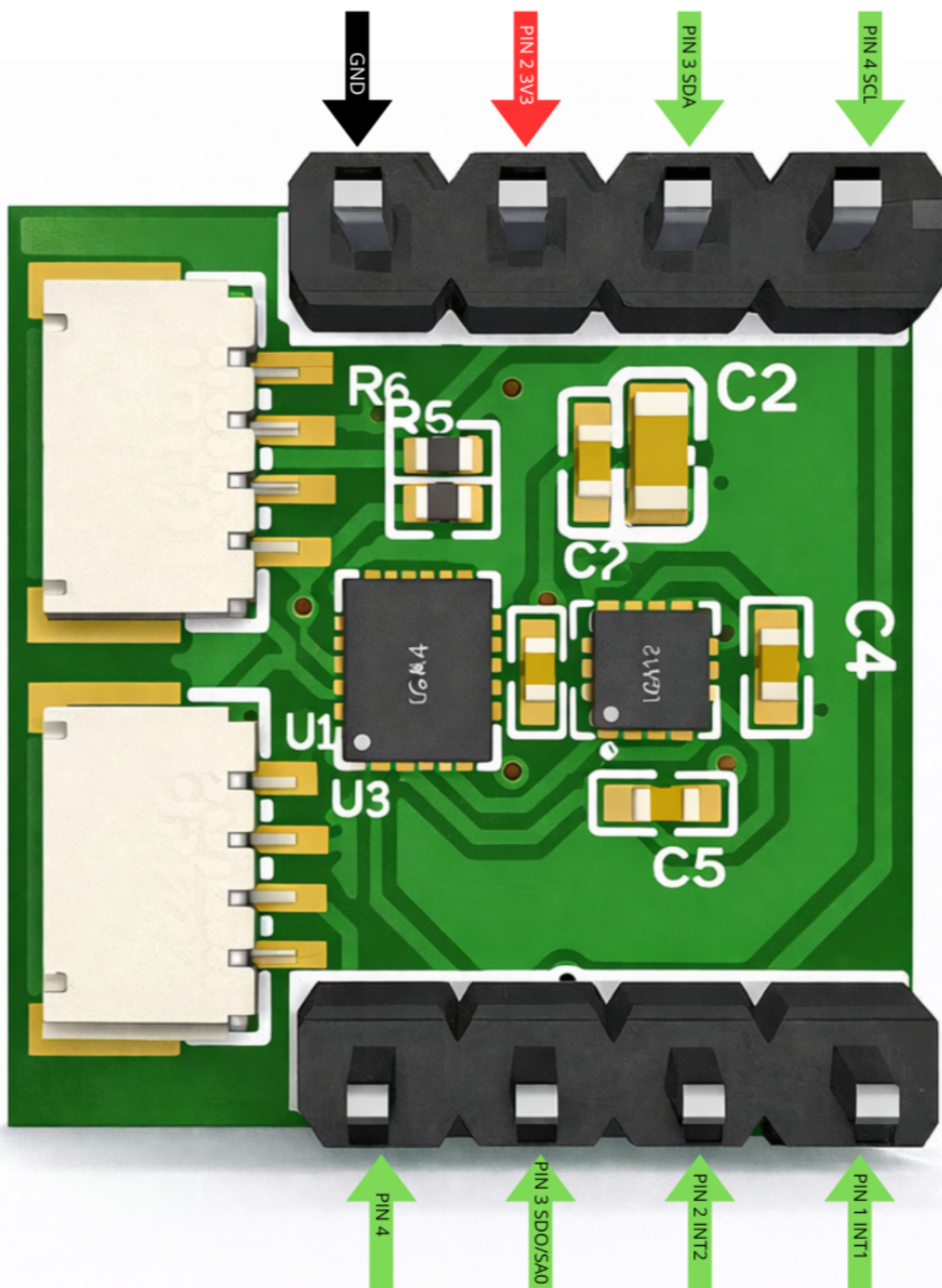


Figure 2: Diagrama de pines de la NOVA_sense

- Desde Thonny puedes escribir scripts que usen la clase I2C para leer registros del sensor y guardar lecturas en archivos en la memoria de la NOVA_pico (por ejemplo data.csv). Thonny permite ver y descargar esos archivos fácilmente.

Resumen práctico: la NOVA_pico actúa como «cerebro» que habla con la NOVA_sense por I2C y almacena o envía los datos a tu computador para su análisis. Aquí nos centramos en cómo conectar físicamente y en las ideas generales para leer datos.

3 Programación con MicroPython

3.1 Herramientas recomendadas

- Thonny: recomendado para empezar con MicroPython; muestra la consola REPL y permite editar/guardar archivos en la placa fácilmente.
- rshell/ampy/mpremote: herramientas para usuarios más avanzados.

3.2 Ver las lecturas en Thonny

1. Conecta la NOVA_pico por USB y abre Thonny.
2. Selecciona el intérprete MicroPython correspondiente a tu placa.
3. Abre la consola REPL (panel inferior): al ejecutar `print()` verás la salida inmediatamente en esa consola; es útil para depurar lecturas del sensor.

3.3 Ejemplos básicos en MicroPython (en Thonny)

Nota: los ejemplos usan la clase I2C de MicroPython. Ajusta los pines `scl` y `sda` según tu placa (en la NOVA_pico suelen ser los pines I2C por defecto).

Ejemplo A — Escanear el bus I2C y leer registro WHO_AM_I (identidad)

```
from machine import I2C, Pin
import time

# Ajusta los pines según tu placa
i2c = I2C(0, scl=Pin(21), sda=Pin(20))

print('Escanear I2C...')
devices = i2c.scan()
print('Dispositivos I2C encontrados:', devices)

# Registro de identidad (WHO_AM_I) indicado para este sensor
WHO_REG = 0x4F

for dev in devices:
    try:
        who = i2c.readfrom_mem(dev, WHO_REG, 1)
        print('Dispositivo', hex(dev), 'WHO_AM_I =', who[0])
    except Exception:
        # no todos los dispositivos responden a ese registro
        pass

time.sleep(1)
```

En la consola de Thonny verás la lista de direcciones I2C detectadas y, si alguno responde al registro WHO_AM_I, su valor.

Ejemplo B — Leer datos crudos del acelerómetro (lectura genérica)

```
from machine import I2C, Pin
import struct
```

```

i2c = I2C(0, scl=Pin(21), sda=Pin(20))

# Reemplaza DEVICE_ADDR por la dirección que encontró tu escaneo
DEVICE_ADDR = 0x6A # ejemplo, puede variar
OUTX_L = 0x28      # registro inicial de datos (ejemplo común)

def read_accel(addr):
    raw = i2c.readfrom_mem(addr, OUTX_L, 6)
    # convertir 6 bytes en tres valores signed 16-bit (little endian)
    x = struct.unpack('<h', raw[0:2])[0]
    y = struct.unpack('<h', raw[2:4])[0]
    z = struct.unpack('<h', raw[4:6])[0]
    # la escala depende del sensor; aquí mostramos los crudos
    return x, y, z

try:
    x, y, z = read_accel(DEVICE_ADDR)
    print('Acelerómetro raw:', x, y, z)
except Exception as e:
    print('Error:', e)

```

Explicación sencilla: los valores crudos son enteros que cambian si mueves el sensor. En Thonny verás los `print()` aparecer en la consola.

3.4 Guardar lecturas en un archivo desde Thonny

Puedes guardar lecturas en la memoria de la NOVA_pico para analizarlas luego:

```

import time

with open('data.csv', 'w') as f:
    f.write('t,x,y,z\n')
    for t in range(100):
        x, y, z = read_accel(DEVICE_ADDR)
        f.write(f"{t},{x},{y},{z}\n")
        time.sleep(0.1)

```

Consejos de depuración:

- Usa `print()` para depurar en la consola de Thonny.
- Si obtienes errores de lectura, revisa que GND esté conectado correctamente y que la alimentación sea 3.3 V.

4 Seguridad y buenas prácticas

Antes de trabajar con la NOVA_sense, ten en cuenta estas recomendaciones para evitar daños al hardware y asegurar lecturas fiables:

- **Alimentación correcta:** usa 3.3 V en los pines de señal y alimentación. No conectes 5 V a los pines I2C o de señal.
- **GND común:** siempre conecta la masa (GND) entre la NOVA_pico (o microcontrolador) y la NOVA_sense antes de alimentar el sistema.
- **Conexiones firmes:** evita cables sueltos; utiliza conectores o headers bien soldados para prevenir desconexiones durante pruebas.
- **Evita campos magnéticos fuertes:** el magnetómetro (LIS2MDL) se altera con imanes o corrientes cercanas — mantén alejada la NOVA_sense de fuentes magnéticas potentes durante calibración y mediciones.
- **Calibración y pruebas iniciales:** realiza un `i2c.scan()` y verifica el `WHO_AM_I` antes de usar los datos; calibra la brújula si vas a usar orientación absoluta.
- **Protección ESD:** manipula la placa en una superficie antiestática o descarga tu energía con una pulsera ESD si trabajas en un entorno sensible.
- **Documenta cambios:** si modificas hardware (jumpers, resistencias), anota los cambios para reproducir pruebas y diagnóstico.

5 Computación Física con NOVA_sense

5.1 ¿Qué es una lectura analógica?

En el mundo digital un pin solo puede estar **encendido** (1) o **apagado** (0). Una lectura analógica es diferente: mide un voltaje que puede tomar **cualquier valor entre 0 V y 3.3 V**, como el brillo variable de una lámpara con regulador. El componente que hace esta conversión dentro de la NOVA_pico se llama **ADC** (*Analog-to-Digital Converter*) — traduce un voltaje real a un número que tu programa puede usar.

¿Cuándo necesitas lecturas analógicas?

- Medir la **posición** de una perilla (potenciómetro).
- Detectar **niveles de luz** con un LDR (fotoresistencia).
- Leer la **temperatura** con un sensor analógico como el LM35 o TMP36.
- Medir **voltajes de batería** o señales de sensores industriales.
- Cualquier magnitud que varíe de forma continua (no solo encendido/apagado).

5.2 Pines ADC de la NOVA_pico

La NOVA_pico (basada en el RP2350A) tiene **4 canales ADC** accesibles y un canal interno:

Canal ADC	Pin GPIO	Función
ADC0	GPIO26	Entrada analógica de propósito general
ADC1	GPIO27	Entrada analógica de propósito general
ADC2	GPIO28	Entrada analógica de propósito general
ADC3	GPIO29	Entrada analógica (también midiendo VSYS/3 en algunas placas)
ADC4	— (interno)	Sensor de temperatura interno del chip

Importante: los pines **GPIO26, GPIO27 y GPIO28** son los que usarás normalmente para conectar sensores analógicos. No confundas estos pines con los de I2C (GPIO20 SDA, GPIO21 SCL) que ya usas para la NOVA_sense.

Resolución: el ADC del RP2350A es de **12 bits**, lo que significa que lee valores de 0 a 4095. Sin embargo, MicroPython los reporta como valores de **16 bits** (0 a 65535) con `read_u16()`, escalando internamente para mantener compatibilidad con otras placas.

5.3 Tu primera lectura analógica — un potenciómetro

Un potenciómetro (perilla giratoria) es el sensor analógico más sencillo: al girarlo, su voltaje de salida cambia suavemente de 0 V a 3.3 V.

Materiales:

- NOVA_pico
- Potenciómetro de 10 kΩ (cualquier valor entre 1 kΩ y 100 kΩ sirve)
- 3 cables dupont

Conexión:

Pin del potenciómetro	Conectar a
Pata izquierda	GND de la NOVA_pico
Pata central (wiper)	GPIO26 (ADC0)
Pata derecha	3V3 de la NOVA_pico

Código — leer el potenciómetro y mostrar en consola:

```
from machine import ADC, Pin
import time

# Crear el objeto ADC en GPIO26 (ADC0)
pot = ADC(Pin(26))

while True:
    # Leer valor crudo (0 - 65535)
    valor_crudo = pot.read_u16()

    # Convertir a voltaje real (0.0 - 3.3 V)
    voltaje = valor_crudo * 3.3 / 65535

    # Convertir a porcentaje (0 - 100%)
    porcentaje = valor_crudo * 100 / 65535

    print(f"Crudo: {valor_crudo:5d} | Voltaje: {voltaje:.2f} V | Posición: {porcentaje:.1f}%")
    time.sleep(0.3)
```

¿Qué verás en Thonny? Al ejecutar este script, la consola REPL mostrará algo así:

```
Crudo: 32750 | Voltaje: 1.65 V | Posición: 50.0%
Crudo: 1200 | Voltaje: 0.06 V | Posición: 1.8%
Crudo: 64800 | Voltaje: 3.26 V | Posición: 98.9%
```

Gira la perilla y los valores cambian en tiempo real. Este mismo principio aplica a **cualquier sensor analógico**.

5.4 Leer un sensor de luz (LDR / fotoresistencia)

Un LDR (**Light Dependent Resistor**) cambia su resistencia según la cantidad de luz que recibe: mucha luz = baja resistencia, oscuridad = alta resistencia. Usamos un **divisor de voltaje** para convertir ese cambio de resistencia en un cambio de voltaje que el ADC pueda leer.

Materiales:

- NOVA_pico
- LDR (fotoreistencia)
- Resistencia de 10 kΩ
- Cables dupont y protoboard

Conexión (divisor de voltaje):

3V3 LDR Resistencia 10kΩ GND

GPIO26 (ADC0)

La unión entre el LDR y la resistencia de 10 kΩ se conecta a **GPIO26**. Cuando hay mucha luz el LDR baja su resistencia y el voltaje en GPIO26 sube; en oscuridad el voltaje baja.

Código — detector de luz con umbral:

```
from machine import ADC, Pin
import time

ldr = ADC(Pin(26))

# Umbral: ajústalo según tu ambiente
UMBRAL_OSCURO = 20000 # por debajo de esto → "oscuro"
UMBRAL_CLARO = 45000 # por encima de esto → "muy iluminado"

while True:
    lectura = ldr.read_u16()

    if lectura < UMBRAL_OSCURO:
        estado = " Oscuro"
    elif lectura > UMBRAL_CLARO:
        estado = " Muy iluminado"
    else:
        estado = " Luz media"

    print(f"ADC: {lectura:5d} → {estado}")
    time.sleep(0.5)
```

Tip: los valores de umbral dependen de tu LDR y de la resistencia que uses. Ejecuta primero sin umbrales, observa los valores en Thonny, y luego ajusta los números.

5.5 Leer temperatura con un sensor analógico (LM35 / TMP36)

Los sensores LM35 y TMP36 generan un voltaje proporcional a la temperatura. La conversión es directa:

Sensor	Fórmula
LM35	Temperatura °C = voltaje × 100 (10 mV por °C)
TMP36	Temperatura °C = (voltaje – 0.5) × 100

Conexión del LM35:

Pin del LM35	Conectar a
VCC	3V3 de NOVA_pico
VOOUT (centro)	GPIO26 (ADC0)
GND	GND de NOVA_pico

Código — termómetro digital:

```

from machine import ADC, Pin
import time

sensor_temp = ADC(Pin(26))

while True:
    adc_val = sensor_temp.read_u16()

    # Convertir a voltaje
    voltaje = adc_val * 3.3 / 65535

    # Convertir a temperatura (fórmula LM35)
    temp_c = voltaje * 100 # 10 mV/°C

    print(f"Voltaje: {voltaje:.3f} V → Temperatura: {temp_c:.1f} °C")
    time.sleep(1)

```

Salida esperada en Thonny:

```

Voltaje: 0.230 V → Temperatura: 23.0 °C
Voltaje: 0.245 V → Temperatura: 24.5 °C

```

5.6 Sensor de temperatura interno del RP2350A

La NOVA_pico tiene un sensor de temperatura **integrado en el chip** (ADC canal 4), sin necesidad de hardware adicional. Es útil para monitorear si la placa se calienta demasiado:

```

from machine import ADC
import time

temp_interna = ADC(4) # Canal 4 = sensor interno

while True:
    lectura = temp_interna.read_u16()
    # Fórmula del datasheet del RP2350
    voltaje = lectura * 3.3 / 65535
    temp_c = 27 - (voltaje - 0.706) / 0.001721

    print(f"Temperatura del chip: {temp_c:.1f} °C")
    time.sleep(2)

```

Este sensor mide la temperatura del **chip**, no del ambiente. Es normal que marque unos grados más que la temperatura ambiental.

5.7 Leer múltiples sensores analógicos al mismo tiempo

Puedes conectar hasta 3 sensores analógicos simultáneamente usando los canales ADC0, ADC1 y ADC2:

```

from machine import ADC, Pin
import time

pot = ADC(Pin(26)) # ADC0 - potenciómetro

```

```

ldr = ADC(Pin(27)) # ADC1 - sensor de luz
temp = ADC(Pin(28)) # ADC2 - sensor de temperatura

print("POT      | LUZ      | TEMP (°C)")
print("-" * 35)

while True:
    v_pot = pot.read_u16()
    v_ldr = ldr.read_u16()
    v_temp = temp.read_u16()

    # Convertir temperatura (LM35)
    temp_c = (v_temp * 3.3 / 65535) * 100

    print(f"{v_pot:5d}      | {v_ldr:5d}      | {temp_c:.1f}")
    time.sleep(0.5)

```

5.8 Guardar lecturas analógicas en un archivo CSV

Para analizar datos después (por ejemplo en una hoja de cálculo), puedes guardarlos en la memoria de la NOVA_pico:

```

from machine import ADC, Pin
import time

adc = ADC(Pin(26))
MUESTRAS = 200      # cantidad de lecturas
INTERVALO = 0.1     # segundos entre cada lectura

with open('lecturas_adc.csv', 'w') as f:
    f.write('muestra,valor_crudo,voltaje\n')
    for i in range(MUESTRAS):
        val = adc.read_u16()
        volt = val * 3.3 / 65535
        f.write(f'{i},{val},{volt:.4f}\n')
        time.sleep(INTERVALO)

print(f'Listo: {MUESTRAS} muestras guardadas en lecturas_adc.csv')

```

Después puedes descargar el archivo desde Thonny (panel de archivos del dispositivo → clic derecho → *Descargar a...*) y abrirlo en Excel, Google Sheets o cualquier programa de hojas de cálculo para graficar los datos.

5.9 Consejos para obtener lecturas estables

- **Cables cortos:** los cables largos captan ruido eléctrico. Mantén las conexiones al ADC lo más cortas posible.
- **Promedio de lecturas:** si los valores fluctúan, toma varias muestras y promédialas:

```

def leer_promedio(adc, n=10):
    suma = 0
    for _ in range(n):

```

```
    suma += adc.read_u16()  
    return suma // n
```

- **Condensador de filtro:** un condensador cerámico de 100 nF entre el pin ADC y GND reduce el ruido eléctrico significativamente.
- **No mezclar señales ruidosas:** mantén los cables analógicos separados de motores o actuadores que generan interferencia.
- **Alimentación estable:** usa la salida 3V3 de la NOVA_pico como referencia; voltajes inestables producen lecturas erróneas.

5.10 Uso de sensores comunes (temperatura, luz, movimiento)

Contenido sobre sensores comunes...

5.11 Actuadores (motores, servos) y consideraciones de potencia

Contenido sobre actuadores...

6 Proyectos de ejemplo

6.1 Semáforo (Traffic light)

Contenido del proyecto semáforo...

6.2 Juego de reacción

Contenido del proyecto juego de reacción...

6.3 Alarma antirrobo básica

Contenido del proyecto alarma...

6.4 Registrador de datos (Data Logger)

Contenido del registrador de datos...

6.5 Ejemplos paso a paso: materiales, esquemas, código

Contenido paso a paso...